Structural Testing Methodology for Deep Neural Networks based on RRAM

Emmanouil Anastasios Serlis[1], Emmanouil Arapidis[1], Theofilos Spyrou[1], Anteneh Gebregiorgis[1], Mottaqiallah Taouil[1,2], Said Hamdioui[1,2], Moritz Fieback[1]
[1]Computer Engineering Laboratory, Delft University of Technology, Mekelweg 4, 2628CD, Delft, The Netherlands
[2]CognitiveIC, Van der Burghweg 1, 2628CS, Delft, The Netherlands

***Abstract*—Compute-in-memory (CIM) AI accelerators based on emerging non-volatile memories like Resistive Random Access Memory (RRAM) integrate memory cells with mixed-signal peripherals to enable energy-efficient, low-latency edge computing. However, these specific circuit modifications introduce unique defects and fault models not found in standard memory, necessitating the development of novel testing approaches. Existing literature focuses either on functionally testing CIM AI accelerators on a system level, which cannot ensure that these circuits are defect-free electrically, or on the accurate circuit-level testing, which is limited to a few memory cells and cannot be scaled to system-level. This work bridges the gap between abstract system-level modeling and limited circuit-level testing by presenting the first structural test development methodology for CIM Deep Neural Networks (DNNs). The methodology first accurately models defects and faults at the RRAM crossbar level and develops effective structural tests to detect them. Next, these test results are propagated to single and multiple DNN layers, allowing to structurally test RRAM-based DNNs. Finally, different sets of test patterns are generated, optimized either for >96% maximum defect coverage or reduced test time up to 3 times of the nominal test set, while keeping >73% defect strength coverage for all unique defects.**

***Index Terms*—DNN Testing, Computation-in-Memory, Structural Testing, RRAMs**

I. Introduction

Computation-in-memory (CIM) AI Accelerators bypass the von-Neumann bottleneck, providing the potential for enhanced computational performance. They can be implemented via analog computations for better energy efficiency in large macros [1]. Such computational structures deploy analog and mixed-signal circuit blocks, such as analog-to-digital (ADC) and digital-to-analog (DAC) converters. In order to combine data and computation in a single module, many of the silicon-proven CIM AI architectures are based on memristive non-volatile RRAM devices [2, 3], which come with unique device-level manufacturing defects, from conductance drift and read disturb faults to wrong filament formation [4, 5]. Because differences between CIM architectures and classical memory modules introduce new faults [6], detecting them requires special test attention, and thus efficient structural test methods for analog CIM AI that account for both data converters and RRAM memory.

Prior research on analog RRAM CIM testing has largely relied on abstract simulation-level fault injection combined with functional test patterns or system-level mitigation techniques [7–9]. For example, Chen *et al.* embed backdoored checksums, enabling real-time fault or degradation detection during inference with minimal overhead. However, the aforementioned approaches typically assume simplified fault models, such as random stuck-at faults (SAFs) and gaussian noise, while neglecting critical physical non-idealities like IR-drop, resistive and time-varying device effects. More rigorous defect modeling efforts have emerged for both standalone memory and CIM architectures [6, 10–12], pivoting from foundational March-style memory tests to SPICE-level defect injection for AI compute cores. In summary, such works identify compute-specific defects and faults, such as faulty logic operations, and develop high-coverage read/write test sequences. Yet, they showcase significant test complexity, and are validated mostly on small memory designs, making them computationally prohibitive for the large crossbar dimensions of modern CIM AI accelerators. This highlights the need for a scalable, structurally-aware testing strategy.

This paper offers a test framework to bridge the aforementioned gap between software and circuit-level reliability-driven and defect-oriented testing. We propose a 3-step structural testing methodology, which starts from SPICE-level RRAM crossbar results and is expanded in order to perform testing for single-layer and multi-layer RRAM DNNs. This paper:

- Proposes a test methodology that offers circuit-level defect realism, alongside efficient system-level testing.
- Analyzes the effect of various types of DNN components, such as activation functions (AFs) and propagation schemes, and proposes test patterns for conventional resistive defects, optimized for maximum defect coverage or reduced test time, while covering all defect locations.
- Provides a more efficient test set for device-aware (DA) RRAM defects compared to state-of-the-art (SOTA).

The remainder of the paper is structured as follows. Section II provides the required background. Section III analyzes the proposed test methodology and its steps. Section IV analyzes the results of the testing methodology. Section V offers comparison of our methodology with SOTA functional and structural tests for RRAMs. Section VI concludes the paper.

II. Background

A. RRAM

RRAM devices, as illustrated in Fig. 1a, employ a metal–insulator–metal (MIM) structure where a thin oxide layer, commonly TiOx or HfOx, is sandwiched between two

(a) RRAM Structure (b) Measurement and simulation model [13] (c) RRAM Resistance Range (d) 1T1R cell (e) 2T2R cell

Fig. 1: RRAM Device Characteristics

electrodes [14]. Operation is governed by the reversible formation of a conductive filament (CF), where a SET voltage drives oxygen ions away to create a low-resistance (LRS) vacancy chain, while a RESET voltage returns the ions to disrupt the filament and restore the high-resistance state (HRS) [15]. The generated I-V curve from such a mechanism is shown in Fig. 1b. Beyond the binary memory states of '1'/LRS and'0'/HRS shown in 1c, RRAM mechanism enables the analog resistance tuning with more resistive states such as High (H), Low (L) and intermediate (I), essential for neuromorphic computing.

In addition to the basic MIM stack, RRAM devices are often part of memory cell structures like one-transistor–one-resistor (1T1R) of Fig.1d and two-transistor-two-resistor (2T2R) of Fig. 1e. The former allows binary multiply-and-accumulate (MAC) operations, while the latter is more versatile, enabling ternary operations via the BLP/BLN inputs (IN) and Ncell_P/Ncell_N weights (W), and thus signed arithmetic INxW operations ([-1,0,1]). Apart from their bit-wise operation flexibility, 2T2R-based CIM Accelerators have demonstrated enhanced performance in literature [2, 16], and showcase a more complex defect detection problem, both in terms of the increased defect locations and signed INxW MAC operations.

B. DNN CIM

Analog RRAM crossbars implement DNNs by encoding weights as conductance levels to execute parallel MAC operations, as shown in Fig. 3a. The system converts digital activations into analog voltages, driving RRAM cells to generate currents proportional to their stored weights. Kirchhoff's law sums these currents along columns for highly parallel vector–matrix multiplication, after which ADCs digitize the output for subsequent processing. The architecture enables energy-and-time efficient parallel MAC operations, despite requiring reliable circuit design to ensure accuracy [17].

Fig. 3b demonstrates the challenges in RRAM-based DNN testing. In particular, we notice that not all DNN locations are ideal for defect detection. That is especially true for analog blocks such as the RRAM 2T2R cell and the accumulated SL current, since their direct measurement would need special equipment or an increased amount of analog I/O test pads. On the other hand, measuring at the ADC output of the crossbar or after the applied activation function (AF) is far more adequate since such results can be stored in-memory and processed afterwards. Next, defect observability is also not possible on the same layer but after 1 or more layers of propagation or even at the DNN output. Such an analysis is suitable in case test circuit infrastructure, such as MBIST controllers and on-chip registers are expensive to setup for each DNN layer.

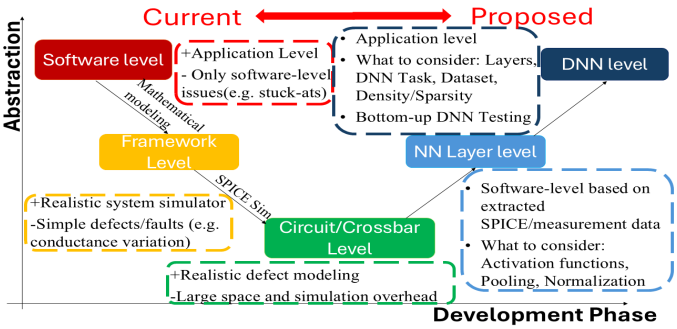

Fig. 2: Proposed Test Development Methodology for RRAM DNNs

III. Testing Methodology

The overall test methodology is presented in Fig. 2. It fills the gap between system-level and circuit-level approaches. Software and framework-level test works offer system-level results, and thus are ranked at the highest level on the abstraction y-axis. However, they depend on abstract defect models, such as stuck-at faults or RRAM conductance variations, which do not include all possible electrical issues in AI compute cores. Such approach correlates with data corruption issues of undefined root causes, and increased debug time [18].

Our methodology starts by performing extensive circuit/crossbar level simulations of the targeted CIM crossbar with injected defects, which range from simple parasitic resistor and capacitor models to cell and device-aware methods [19, 20], for accurate and efficient defect modeling. Such an approach, despite the long simulation overhead, can be done in parallel to the design process of an RRAM-based AI chip, and be kept as a static defect database, as long as the RRAM-based crossbar design remains the same in future iterations.

In order to proceed further right in the test development phase x-axis, the crossbar results are combined at the software level with common DNN primitives, such as ReLU, sigmoid and tanh AFs, normalization layers, and pooling techniques [21], for single-layer and DNN testing, because such operations alter the initial crossbar outputs and therefore affect optimal test pattern generation and detection metrics. For instance, a case of scaling down an 8-bit ADC code with a low-resolution AF would lead to the collapse of multiple values to the same one. Design-for-testability (DfT) constraints and DNN tasks, such as classification or regression, also demand control and task-based analysis to optimize testability.

A. Crossbar Testing

The circuit-level test analysis includes thorough defect & fault modeling, as well as crossbar-level test development solutions, as shown in the flowchart of Fig. 4. The first phase, defect modeling, involves identifying potential defects in both

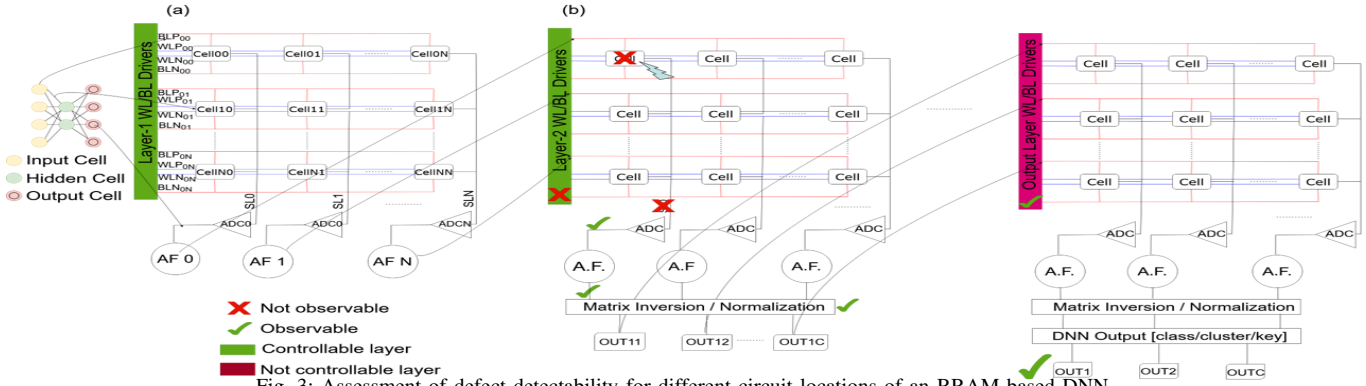

Fig. 3: Assessment of defect detectability for different circuit locations of an RRAM-based DNN.

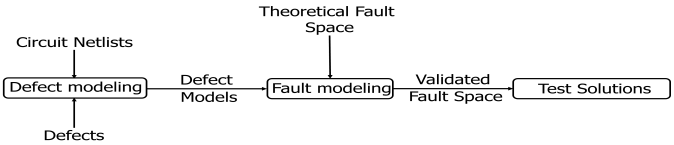

Fig. 4: Test Development Methodology for RRAM Crossbar Array [11]

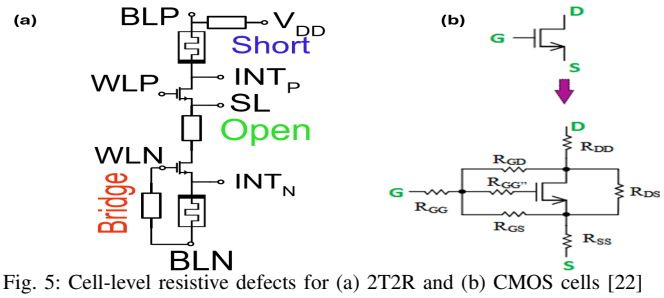

Fig. 5: Cell-level resistive defects for (a) 2T2R and (b) CMOS cells [22]

TABLE I: Device-aware defect modeling [13]

Defect Symbol	Description	Defective Range
OR	Over Reset	$N_{min} \in [0.001 - 0.01] \cdot 10^{26}\,\mathrm{m}^{-3}$
ID	Ion Depletion	$N_{min} \in [1.5 - 4.5] \cdot 10^{24}\,\mathrm{m}^{-3}$
IUSF	Undefined State Fault	$N_{max} \in [1.5 - 4.5] \cdot 10^{24}\,\mathrm{m}^{-3}$
OF	Over Forming	$r_{det} \in [70 - 100]\,\mathrm{nm}$
UF	Under Forming	$r_{det} \in [10 - 30]\,\mathrm{nm}$

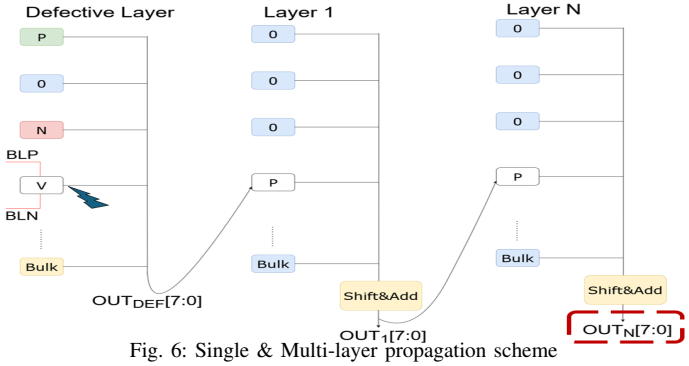

Fig. 6: Single & Multi-layer propagation scheme

the memory array and peripheral circuits, such as ADCs and DACs, which serves as the foundation for constructing corresponding defect models. During the second step of fault modeling, these models are integrated into the initial crossbar netlist and simulated to determine which faults from the theoretical space actually manifest in the circuit and require testing, leading to a validated fault space. The third step involves developing tests for validated faults such as DfT circuits or pattern generation algorithms.

In this work, 2T2R defect injection is based on the linear resistive RRAM and CMOS defect models presented in Fig. 5 [22]. For each unique defect position, a scale of $10\,\Omega$ to $5\,\mathrm{M}\Omega$ on 10 logarithmic steps is used to simulate all targeted defects. In our setup, defects are injected separately for RRAM and CMOS devices, assuming one resistive defect per component at a time. Apart from resistive defects, our simulation setup incorporates all RRAM device-aware ones, with the RRAM defect modeling parameters shown in Table I. The inclusion of DA defects comes as a result of the lack of analysis of DA defect effects on DNN workloads, since they have been mostly characterized at device and circuit-level [13].

B. Neural Network Single-Layer Testing

After structural testing of the single RRAM-based CIM core, the next step includes transforming the aforementioned core to a single-layer neural network. This revolves around the variety of AFs utilized in DNNs, with three main AFs being considered in this work: the piece-wise linear AF ReLU, as well as the non-linear AFs sigmoid and tanh ones. NN layer detectability is primarily affected by AF fidelity, specifically the Lookup Table (LUT) size, as hardware AFs typically rely on LUT-based implementations [23]. Since the AF can affect the initial layer output, it is important to simulate the expected test results for various AF configurations.

C. Multi-layer Testing

After performing single-layer structural testing, which is based on realistic circuit defects, the final step is to predict the testing accuracy for a multi-layer RRAM-based DNN. The main goals are to quantify the effect of various defects on each layer, compare which layers are best for input pattern injection, as well as find the best point to observe the output code based on the initial defect location. The single and layer-to-layer propagation schemes are shown in Fig. 6. We apply all INxW product combinations to the first layer, which contains the defective cell. Subsequent layers use neutral weights plus one positive-weighted cell driven by the previous layers' ADC output, which is necessary because a neutral weight would force all propagated inputs to zero. For the single-layer defect effect, we observe the defective layer's ADC output code, $\mathrm{OUT}_{\mathrm{DEF}}$. For propagation on the non-defective layers, we employ a software shift-and-add scheme, common in multi-bit I/O RRAM analog compute macros [24], since inputs are passed in a bit serial fashion between connected layers. Finally,

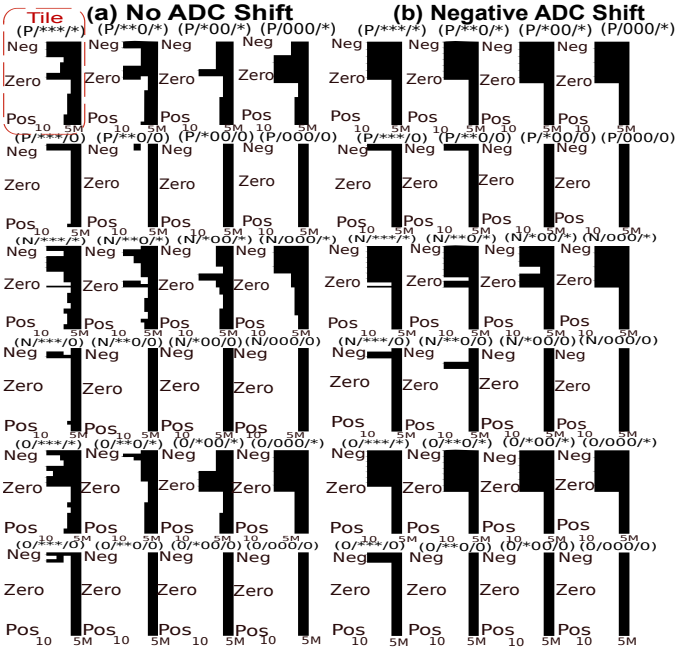

Fig. 7: Output fault maps for observing ADC crossbar output with (a) no ADC shift (b) negative ADC shift for a VCO ADC internal bridge defect. Defect detection and lack of it are shown in white and dark color respectively.

for multi-layer estimation, defective and non-defective results are compared based on the Nth-layer output, $\mathrm{OUT_N}$.

IV. Application of the Testing Methodology

The testing methodology is applied on a crossbar array of 8 rows and 2 columns, following the structure shown in Fig 3. Collapsed parallel input patterns are applied to the column of the defective victim cell, where 3 unique 2T2R RRAM neighbors provide discrete INxW states ([-1, 0, 1]), and the remaining 4 column neighbors are assigned a common INxW bulk value. The Word Lines (WL) operate at 1.6 V, whereas different BLP/BLN voltages are utilized per operation; in particular, for inference/read of positive inputs, voltages are set at BLP=850mV and BLN=250mV, with the reverse being true for negative inputs, whereas neutral INs are realized via BLP=BLN=550mV. RRAM SET operation is done with 1.25V and RESET with 2.5V. During inference, the output SL current of each column is read by an 8-bit voltage-controlled oscillator (VCO) ADC [25]. To accurately model the RRAM mechanism and its defects, we apply the physics-based JART VCM v1b RRAM model [26]. Defective and non-defective netlists are simulated in Cadence Spectre, with SPICE-level results extended to the neural network level via custom Python software. This software implements DNN functionalities, such as AFs, multi-layer propagation, residual connections, and DNN tasks, to bridge system-level complexity with low-level defect data, enabling a comprehensive evaluation of the simulation setup by integrating the above modeling techniques.

A. Crossbar-level Test Results

The fault maps presented in Fig. 7 are organized into 24 distinct memory write background tile derived from combinations of the victim cell, the bulk and the 3 unique

TABLE II: Mapping of write background codes to column weight values

Write background Symbol	Weight Description	Applied cell
P	Positive weight	Victim
N	Negative weight	Victim
0	Neutral weight	Victim, Neighbors, Bulk
*	Positive or negative weight	Neighbors, Bulk

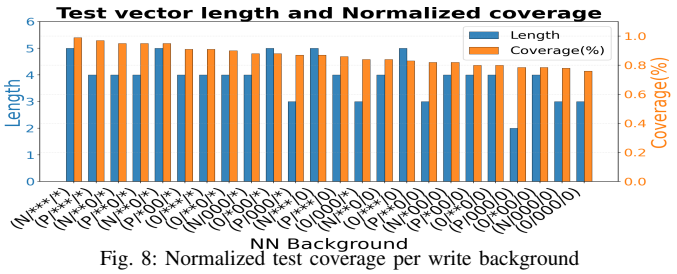

Fig. 8: Normalized test coverage per write background

neighbor cells. Within each tile, as the (P/***/*) highlighted in Fig. 7a, the horizontal axis represents increasing resistive defect strength (from 10Ω to 5MΩ), while the vertical axis arranges crossbar column inputs from the most negative to the most positive column pattern to visualize input-dependent effects. Neighbor & bulk cells are restricted to positive and neutral weight initializations, excluding the negative case since equivalent negative dot products are generated via BLPs/BLNs of the 2T2R cell, as IN=-1 (BLP=250mV/BLN=850mV) and W=+1 (Ncell$_P$=LRS/'1' & Ncell$_N$=HRS/'0'), thus removing the IN=+1/W=-1 neighbor cell simulation. This results in a categorization starting from the non-neutral $(\mathrm{P}, *, *, *)$ top-left tile to the all-neutral $(0, 000, 0)$ bottom-right one using a (V/Vi/N/B) notation detailed in Table II. V is the victim cell weights, Vi is the victim cell initial state (before any read or write operations), N are the weights of the 3 distinct neighbors, B is the equivalent weight of the bulk cells. For instance, (P/N/**0/0) denotes a write background where the defective cell has been switched from negative to positive, has 2 out of 3 distinct neighbors non-neutral (positive or negative) and the 4 bulk cells all at neutral weight. The visualization of an ADC VCO resistive defect case in Fig. 7 reveals critical testing insights, as negative inputs in most write backgrounds fail to detect induced open defects as shown in Fig. 7b. This inadequacy is evident in backgrounds like (0/**0/0) and (P/*00/*), where negative inputs yield low coverage compared to the near-full one achieved in Fig. 7a.

We categorize the performance of the 24 write backgrounds, with the single-layer statistics shown in Fig. 8. We notice that the write backgrounds with the highest average defect coverage would belong to the $(\mathrm{P}, *, *, *)$ and $(\mathrm{N}, *, *, *)$ categories. In other words, they provide maximum test coverage despite the increased test vector lengths (4 and 5 respectively). Test patterns from such backgrounds should be preferable since they require a single initialization of all crossbar weights, which would lead to lower test times. Referring to the test vector cost per write background, patterns from backgrounds $(\mathrm{P}, 000, 0)$, $(\mathrm{N}, 000, 0)$ and $(0, 000, 0)$, with an increased number of neutral weights $N1$-$N3$, seem to be the most cost effective during inference at the expense of low test accuracy. However, the final test time should be hindered by the need to switch from P/N weights to neutral ones,

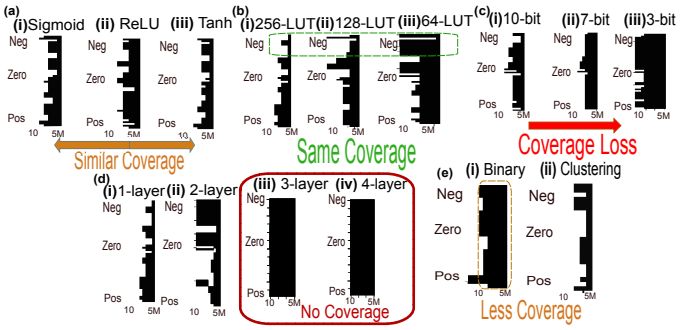

Fig. 9: Fault map examples for different (a) AFs (b) Sigmoid A.F. LUT sizes (c) Sigmoid A.F. bit resolution (d) layer propagation and (e) DNN Task

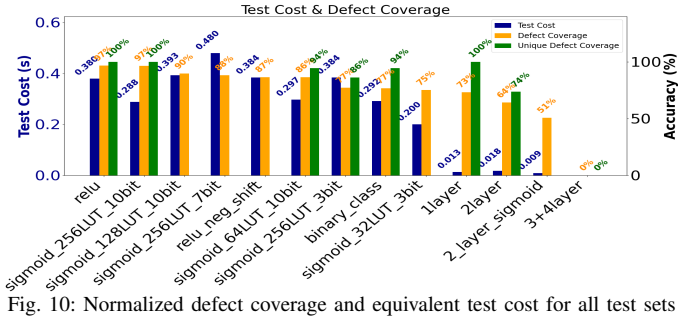

Fig. 10: Normalized defect coverage and equivalent test cost for all test sets

specifically in the $(\mathrm{P}, 000, 0)$ and $(\mathrm{N}, 000, 0)$ cases.

B. Single-Layer Test Results

In order to extend to single-layer results, we first quantify the effect of adding one of the sigmoid, ReLU or tanh AFs. Fault maps for a single 2T2R defect case are shown in Fig. 9a. For the sigmoid and tanh cases, 256 unique integer LUT values are included, to match the resolution of an 8-bit ADC. It is clearly shown that all 3 AFs have very similar defect coverage, both in terms of absolute detectability as well as number of available test patterns. The only difference refers to the exclusion of some of the most negative and positive test patterns in the sigmoid and tanh cases, due to the increased non-linearity of those AFs near their rails.

Analysis for different AF settings is provided via the fault maps of Figs. 9b and 9c. For those cases only the sigmoid case has been included, since the tanh one can be obtained as the derivative of the sigmoid. From these fault maps, we conclude that lowering the AF bit resolution has a greater effect than decreasing the LUT size even to just 64 elements. That is because a decrease even in decimal resolution point (e.g. from 3 to 2 decimal points) automatically reduces the LUT size to a maximum of 100 elements. Such an observation makes us realize that low-LUT AFs could keep higher defect-oriented test accuracy, in contrast to a low-bit resolution AF.

C. Multi-Layer Test Results

For NN-level structural testing, we start by determining the effect of the layer after which we observe the output code, with such a fault map being the one of Fig. 9d. We observe that each propagation layer diminishes available test patterns, with even 1-layer cases yielding significantly fewer test inputs. Consequently, ensuring testability requires observing the output code either immediately or at most one layer after the defective RRAM crossbar. Such structural constraint, however, introduces area and complexity overhead.

Finally, the DNN task also plays an important role, in case we test only at the output of the DNN. Fig. 9e compares single defect fault maps for a computational clustering vs classification task. For this fault map, we utilized the sigmoid AF as a 2-class binary classification threshold, where value above threshold 0.5 lead to class output 1 and values below that threshold lead to class output 0. As expected before simulations, the binary classification leads to much lower defect coverage, since it collapses the high resolution space of the neural network output in just 2 values.

D. Test Cost Analysis

For quantifying structural RRAM-based DNN testing, we have to compare, for all DNN components and types, how many resistive defect cases and defect strength values can be detected, as well as the equivalent test time. Fig. 10 presents the structural test results for all 14 single and multi-layer RRAM DNN cases. We notice that the high resolution single-layer sigmoid AF offers almost the same coverage as the ReLU case. All other cases of adding a sigmoid before the measurement point result in degrading test coverage, due to the decrease in LUT size and resolution, which leads to fewer unique LUT values than those of an 8-bit ADC.

Next, we observe that multi-layer propagation and binary classification degrade performance by masking detectable defects, making such test schemes undesirable. Notably, the 3 and 4-layer propagation schemes lead to zero coverage, since the initial numerical difference gets masked on the neutral-heavy propagation scheme of Fig. 6. Furthermore, test latency typically increases in middle-coverage setups. For example, low-fidelity sigmoid configurations, such as the 64LUT_10bit and the 256LUT_3bit ones, are inefficient because they require more patterns, and thus test time, than the nominal ReLU case, while lowering detectability. In contrast, the layer-by-layer scheme, despite the low defect accuracy, guarantees 100% unique defect coverage at a lower test time than the ReLU/sigmoid cases, due to less inference and write operations, as shown in the listed test sequences in Table III. Overall, we could use such a bi-layer scheme as a fast-scan, full-coverage test and keep the more timely ReLU/sigmoid tests for detecting more defect strengths per location.

V. Comparison with SOTA

A. Comparison for conventional defects

Comparison of functional and structural test methodologies for RRAM-based memories and DNNs is provided on Table III. The listed test patterns follow the (V/N/B) notation from above. We generate test patterns by solving an Integer Linear Programming (ILP) problem via Python's SciPy [27], in order to minimize test length and memory write overhead while maximizing defect coverage. We observe that, between our proposed tests, a tradeoff exists between the ReLU and the high-resolution sigmoid, since the further provides the lowest test cost time, whereas the latter has the highest defect

TABLE III: Test cost comparison for an assumed 128x128 array.

Test	Patterns	Type	Write/Read	Time(s)	Coverage
ReLU	(+1-1:-1+1:+1+1-1-1) (+1-1:00:+1-100) (+1+1:00:+1000) (+1-1:+1-1:-1-1-1-1)	Structural	4N/4N	0.384	**96.7%**
Sigmoid Nominal	(+1-1:00:-1-1-10) (+1-1:+1-1:-1-1-1-1) (+1+1:00:+1+100)	Structural	3N/3N	0.288	96.5%
Sigmoid (100 LUT)	(-1+1:-1+1:0000) (+1-1:-1+1:-1-1-1-1) (+1-1:00:-1-1-10) (+1+1:-1+1:+100+1) (+1-1:00:+1+1+10)	Structural	5N/5N	0.48	88.2%
1-layer	(-1+1:-1+1:000-1) (-1+1:00:000+1) (-1+1:+1-1:000-1)	Structural	1N/3N	**0.105**	73.8%
Low-cost Test [8]	-	Functional	0N/300N	**0.03**	84%
Analog BIST [7]	-	Functional	0N/4000N	0.4	91.9%
March-W (1T1R) [10]	-	Structural	9N/8N	0.86	92%

coverage, but at a 50% test time increase. At the same time, the 128-LUT size sigmoid is too inefficient due to increased test latency and decreased coverage. Finally, the layer-by-layer scheme is the most efficient, while covering all unique defects as shown in Fig 10, but at a lower defect strength coverage.

When comparing our proposed structural tests with the literature ones, our methodology partially lacks behind the low-cost test work of [8], which has the lowest test time at an adequate coverage (>80%). However, all the provided functional tests are based on abstract fault models and thus their results are not as detailed and realistic as in the structural tests of our work and March-W [10], which has almost double the test delay, due to an increased amount of write operations.

B. Comparison for unique defects

Unlike conventional defects that inference-based operations, i.e. patterns that require no new memory re-writes, cover adequately, DA testing requires exploring write operations to ensure defect coverage. While previous work on RRAM DA testing [13] included all 0w0, 1w1, 1w0, and 0w1 operations for such purpose, we test OF and UF defects in the context of an analog RRAM CIM Accelerator, applying high BLP/BLN inference voltages up to 850mV, which correlates directly with changes in the conductive current. Consequently, we replace the unnecessary 0w0 and 1w1 operations of [13] for forming defects with inference-only operations. After solving the ILP problem for the DA defect-input space, we keep only the 1w0 and 0w1 patterns for the remaining RRAM defects in the form of parallel column test patterns $\{$**(-1+1:+1-1:+1-1:+1-1-1-1)**, **(+1-1:-1+1:+1-1:+1-1-1+1)**$\}$. These specific write operations detect OR, ID, and IUSF defects, which equate to changes in maximum and minimum oxide concentrations. Notably, while all non-victim cells in the test set remain in a constant positive or negative state, only the victim cell changes its INxW states.

VI. Discussion & Conclusion

This paper offered a structural testing framework for RRAM-based CIM AI Accelerators, which incorporates low-level SPICE data integrated into high-level neural network structures. This approach finally bridges the gap between circuit-level structural tests and software-level functional ones into one unified methodology. Future aspects include:

Addition of more AI models: This work considered only feedforward DNNs for classification and clustering, but full methodology development should be included other model types, such as attention-based architectures and LSTMs.

Full DNN Workload addition: Because our methodology includes all crossbar input combinations, dataset effects and network density/sparsity are excluded, which would modify the currently proposed test patterns.

DfT architectures: Novel circuit-level modifications, such as DACs for cell trimming, multi-level current references, and advanced VCO-ADC timing schemes, could enhance testability by enabling faster one-shot testing or higher coverage of hard-to-detect defect strength values.

References

[1] J. Sun *et al.*, "Analog or digital in-memory computing? benchmarking through quantitative modeling," in *ICCAD*, 2023.

[2] Q. Liu *et al.*, "33.2 a fully integrated analog reram based 78.4tops/w compute-in-memory chip with fully parallel mac computing," in *ISSCC*, 2020, pp. 500–502.

[3] C.-X. Xue *et al.*, "Embedded 1-mb reram-based computing-in- memory macro with multibit input and weight for cnn-based ai edge processors," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 1, 2020.

[4] L. B. Poehls *et al.*, "Review of manufacturing process defects and their effects on memristive devices," *ACM JET*, 2021.

[5] M. Fieback *et al.*, "Defects, fault modeling, and test development framework for rrams," *J. Emerg. Technol. Comput. Syst.*, Apr. 2022.

[6] M. Fieback *et al.*, "Testing scouting logic-based computation-in-memory architectures," in *ETS*, 2020.

[7] A. K. Mishra *et al.*, "Built-in functional testing of analog in-memory accelerators for deep neural networks," *Electronics*, 2022.

[8] K. Ma *et al.*, "Efficient low cost alternative testing of analog crossbar arrays for deep neural networks," in *ITC*, 2022, pp. 499–503.

[9] C.-Y. Chen *et al.*, "On-line functional testing of memristor-mapped deep neural networks using backdoored checksums," in *ITC*, 2021.

[10] Y. Luo *et al.*, "A high fault coverage march test for 1t1r memristor array," in *EDSSC*, IEEE, 2017.

[11] E. A. Serlis *et al.*, "Structural testing of a rram-based ai accelerator core," in *2025 IEEE European Test Symposium (ETS)*, 2025, pp. 1–6.

[12] Y. .-. Yang *et al.*, "Fault modeling and testing of rram-based computing-in memories," in *ITC-Asia*, 2022.

[13] H. Xun *et al.*, "Robust design-for-testability scheme for conventional and unique defects in rrams," in *ITC*, 2024.

[14] H.-S. P. Wong *et al.*, "Metal–oxide rram," *Proceedings of the IEEE*, vol. 100, no. 6, pp. 1951–1970, 2012.

[15] N. Jain *et al.*, "Resistive random access memory: Materials, filament mechanism, performance parameters and application," in Oct. 2022.

[16] L. Wang *et al.*, "Efficient and robust nonvolatile computing-in-memory based on voltage division in 2t2r rram with input-dependent sensing control," *IEEE TCAS II*, vol. 68, no. 5, 2021.

[17] A. Gebregiorgis *et al.*, "An overview of computation-in-memory (cim) architectures," in *DAECS*. Cham, 2024.

[18] E. J. Marinissen *et al.*, "Silent data corruption: Test or reliability problem?" In *2024 ETS*, pp. 1–7.

[19] X. Xhafa *et al.*, "Sram periphery testing using the cell-aware test methodology," *IEEE TCAD*, 2025.

[20] M. Fieback *et al.*, "Device-aware test: A new test approach towards dppb level," in *ITC*, 2019, pp. 1–10.

[21] M. Vakalopoulou *et al.*, "Deep learning: Basics and convolutional neural networks (cnns)," in *ML for Brain Disorders*. 2023, pp. 77–115.

[22] M. Sekyere *et al.*, "Operational amplifiers defect detection and localization using digital injectors and observer circuits," *Electronics*,

[23] Y. Xie *et al.*, "A twofold lookup table architecture for efficient approximation of activation functions," *IEEE TVLSI*, 2020.

[24] X. Yang *et al.*, "Retransformer: Reram-based processing-in-memory architecture for transformer acceleration," in *ICCAD*, 2020, pp. 1–9.

[25] A. Singh *et al.*, "A 115.1 tops/w, 12.1 tops/mm2 computation-in-memory using ring-oscillator based adc for edge ai," in *AICAS*, 2023.

[26] F. Cüppers *et al.*, "Exploiting the Switching Dynamics of HfO_2-based ReRAM Devices for Reliable Analog Memristive Behavior," *APL*,

[27] P. Virtanen *et al.*, "Scipy 1.0: Fundamental algorithms for scientific computing in python," *Nature methods*, vol. 17, no. 3, 2020.